

Lesson 5.2 — A Worked System-Design Walkthrough with AI

Lesson 5.2 — A Worked System-Design Walkthrough with AI

A 60-minute system-design interview, narrated end-to-end, with AI integrated where it helps and excluded where it doesn't. Use this as a reference for any “design X” interview prompt.

By the end of this lesson:

- You will have a 60-minute system-design walkthrough you can mimic on any prompt.
 - You will know exactly when to ask “may I sanity-check this with the AI?” and when not to bother.
 - You will have memorable lines for each phase of the interview.
-

1. The problem

Pretend the interviewer says:

“Design a fantasy sports application for one million users. Live scoring, leaderboards, news feed, social features. You have 60 minutes. AI tools are permitted. Walk me through your thinking.”

That is a deliberately broad prompt. Most candidates start drawing boxes. **You start by narrowing.**

2. The 60-minute layout

Here is the breakdown we'll narrate:

Minutes	Phase	Goal
0–5	Clarify	Pin down scope, scale, constraints
5–10	Functional requirements	What the system does
10–15	Non-functional requirements	What the system promises
15–25	High-level design	The boxes-and-arrows
25–40	Deep dives	Two or three components in detail
40–50	Trade-offs and risk	Where this design is fragile
50–60	Wrap and Q&A	Summary + interviewer questions

Memorize the column 1 minute-buckets. Within each bucket, you have a script.

3. Phase 1 (0–5 min) – Clarify

What you say

"Before I draw anything, can I confirm the scope? When you say 'one million users,' is that registered total, monthly actives, or daily peak? And by 'fantasy sports' – are we talking pick'em prediction games, or full-roster draft fantasy with trades and waivers? And one more: is there real money on the platform, or is this a free social product?"

Three questions. Each one shifts the design materially:

- *DAU vs total* changes the throughput sizing by 30x.
- *Pick'em vs draft fantasy* changes the data model by an order of magnitude.
- *Real money vs free* changes the entire compliance and reliability story.

The interviewer's likely answer

"DAU 200k, peak 500k. Pick'em-style, like the snooker app you may have seen. No real money – it's a community product."

What you do next

Write the constraints on the board:

SCOPE

- 1M registered, 200k DAU, 500k peak concurrent
- Pick'em fantasy (pick winners, not roster management)
- Free, social, community
- Live scoring, leaderboards, news feed, social features

You just demoed: **clarification before design**. That alone is in the top quartile of candidates.

Where the AI does NOT help

You **don't** ask the AI clarifying questions. You ask the *interviewer*. The clarification phase is human-to-human. Reaching for the AI here would signal you don't know who to talk to.

4. Phase 2 (5–10 min) – Functional requirements

What you say

"OK, given that scope, let me list the functional requirements. I'll bullet them and you can tell me if I miss anything."

Then list, out loud, while writing:

FUNCTIONAL

1. Auth – sign up, log in, email or social
2. Pick submission – user picks winners for upcoming matches
3. Live scoring – when a match completes, scores update for affected users
4. Leaderboards – global and friend-group, top N by total points
5. Match feed – users see today's matches, results, schedule
6. Social – friends, follow, comments on user picks
7. Notifications – push or email when a match user picked completes

Pause, look up. *"Anything I missed?"*

The interviewer might add: *"Yes – admin tooling to enter results."* Add it.

Where the AI helps

You do not ask the AI here either, *but* — if you are 50 minutes in and forgot something, you could quickly prompt:

“Quick sanity check — for a fantasy sports app like the one I described, what’s a common feature people forget? One-line answer.”

That’s a legitimate use of an LLM during a design hour. **Don’t lean on it; do reach for it once when you’re stuck.**

5. Phase 3 (10–15 min) — Non-functional requirements

What you say

“Now the non-functional side. These shape the architecture more than the features do.”

NON-FUNCTIONAL

- Latency: leaderboard read < 200ms p95
- Latency: pick submission < 500ms p95
- Live scoring: visible within 5 seconds of result entry
- Availability: 99.9% (so no database that takes 10 minutes to fail over)
- Consistency: leaderboards can be slightly stale (eventual is fine); picks must be strictly consistent (no double-submits, no lost picks)
- Cost: hobby-tier first year, scale-tier when DAU > 50k
- Compliance: GDPR (EU users), age 13+ (children's privacy)

Look up. *“Are these reasonable, or do you want me to push any of them harder?”*

This sentence — **“are these reasonable, or do you want me to push any harder?”** — is one of the highest-leverage one-liners in any system-design interview. It invites the interviewer into the conversation as a peer rather than as a judge.

Where the AI helps

You *could* sanity-check your latency assumptions with the LLM:

"For a leaderboard query over a million users with score sorting, what's a realistic p95 latency on a single-node Postgres?"

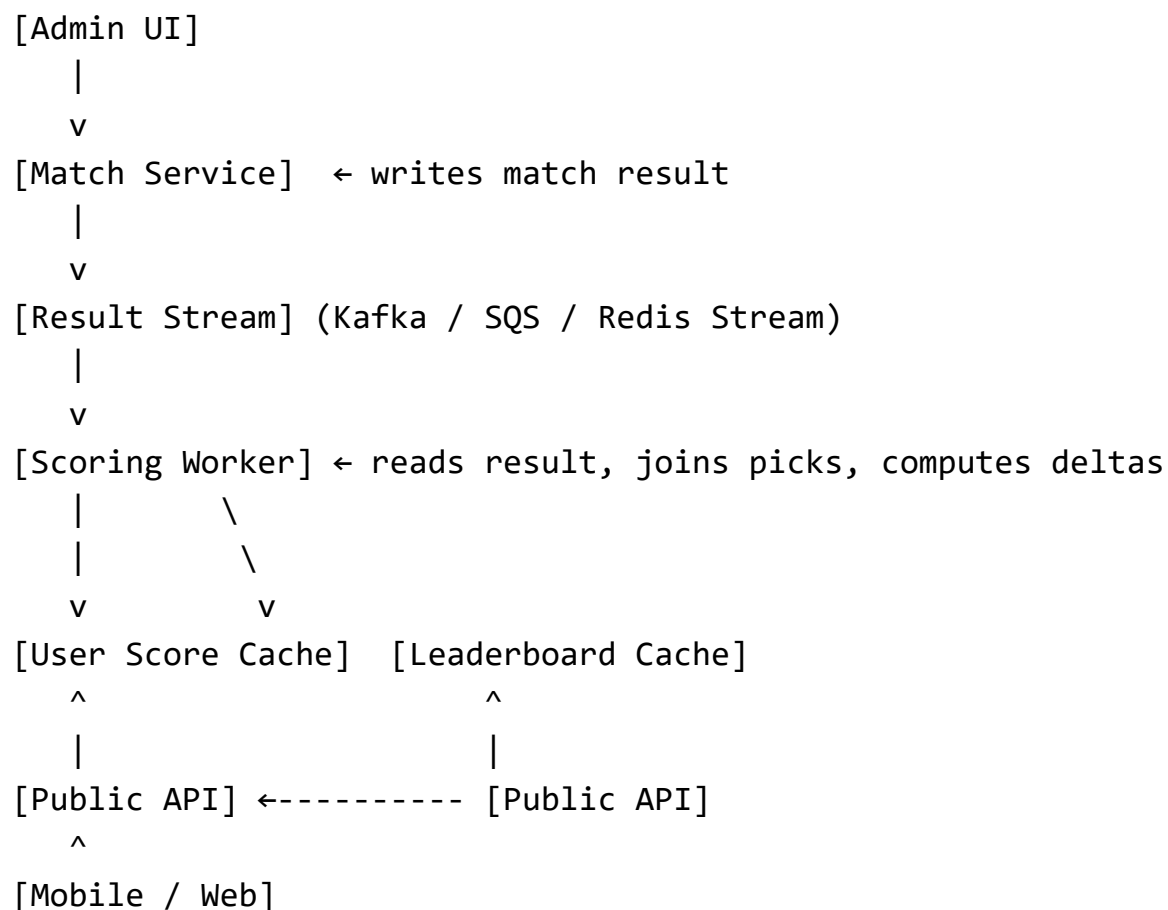
But honestly, by the time you're in a senior interview you should have these numbers in your head. Ask only if you genuinely don't know.

6. Phase 4 (15–25 min) – High-level design

What you say

"OK, the boxes-and-arrows. I'll start with the data flow for the most performance-critical path – live scoring – and work outward."

Sketch:



Narrate while you draw:

"Match result enters via the admin UI. We publish it to a stream. A scoring worker

reads from the stream, joins picks for that match, computes deltas, and updates two caches: per-user score and the global leaderboard. The public API reads from the caches; it doesn't hit the source-of-truth Postgres for hot reads."

What only humans do in this phase

- **Choose** the stream technology. The AI can list Kafka vs SQS vs Redis Streams trade-offs. You pick — based on your team's familiarity, your AWS spend, your latency budget.
- **Choose** the cache topology. Redis cluster? In-process? Edge cache? Each one has consequences for failure modes and dev experience.
- **Decide** which writes go to the stream vs which are synchronous. Pick submission must be synchronous (user expects confirmation). Score updates can be async.

These three decisions are all yours. The AI did not make any of them. The AI could *help* you list trade-offs if you asked it to.

The senior move — narrate the alternatives

When you draw the stream, say out loud:

"I picked a stream because the scoring fan-out is one match → many users. We could do this synchronously — write to the match table, fire a stored procedure that updates all affected user scores. That works at smaller scale. At 500k peak and live scoring under 5 seconds, the synchronous version starts queueing on database connections. Stream gives us back-pressure and lets the scoring worker scale independently. The cost is eventual consistency between match-result and leaderboard, which we said is acceptable in non-functional requirements."

That paragraph is gold. **It shows: you considered an alternative, named the trade-off, and tied the choice back to a stated requirement.** That is exactly the texture of a staff-level interview.

7. Phase 5 (25–40 min) — Deep dives

The interviewer will pick one or two components and say *"go deeper."*

"Tell me more about the leaderboard."

What you say

"OK. The leaderboard has two read patterns: global top-N and friend-group top-N. Both need to be fast and most of the time slightly stale is fine.**

For the global leaderboard, I'd use a Redis sorted set. Score is the user's total points; member is the user ID. ZRANGE for top-N. Fast, capped memory, easy to update incrementally. Updated by the scoring worker on every score change.

For friend-group leaderboards — that one's trickier. We have potentially thousands of overlapping groups. Two strategies:

One, materialize each group's leaderboard as its own sorted set. Updates fan out: a score change goes to global plus every group the user belongs to. Reads are $O(N)$. Writes scale with how many groups a user is in.

Two, compute on read. Read the user IDs from the group, ZSCORE each one, sort, return. Reads are $O(N)$ at API time. Writes only touch global.

I'd start with option two — compute on read — because friend groups are typically small (< 20 users) and the read-time cost is bounded. If groups grow large or read latency suffers, we move to option one. The migration is local — only the leaderboard service changes."

Why this is interview-grade

- You named two strategies.
- You picked one with a reason rooted in expected scale.
- You stated the conditions under which you'd switch.
- You noted that the migration is local — meaning your boundary placement was good.

That last point — *"the migration is local"* — is staff-level reasoning. It says: I designed a boundary, and if reality forces a redesign, the redesign respects the boundary.

Where the AI helps in deep dives

This is the phase where you might genuinely reach for the LLM:

“Quick check – Redis ZRANGE on a sorted set of 1M members, what’s the latency for top-100?”

That’s a real number you might want to verify. Ask the AI if you don’t know. **Naming what you’re checking** is a senior signal – verifying numbers you forgot is fine; pretending to know them is not.

8. Phase 6 (40–50 min) – Trade-offs and risk

What you say

“With time left, let me name the parts of this design I’d most worry about and how I’d mitigate each.”

RISKS

1. Stream backlog during a Sunday-afternoon match storm
2. Cold-cache problem on Redis restart
3. Leaderboard-staleness vs user expectation
4. Admin-UI single source of truth for match results
5. GDPR right-to-be-forgotten with denormalized score data

For each, narrate the mitigation:

“One – stream backlog. Risk: 50 matches finish in a 5-minute window, scoring worker can’t keep up, leaderboards lag by minutes. Mitigation: scoring worker is horizontally scalable, autoscale on queue depth. Bounded concurrency, idempotent processing on the worker. Backup plan: a kill switch that pauses live scoring and shows ‘updating soon’ if the queue exceeds a threshold.

Two – cold cache. If Redis restarts, the leaderboard is empty until rebuild. We’d need a cold-start procedure: pull all users’ totals from Postgres, ZADD into Redis. Takes minutes. During that window, leaderboards are stale. Could we tolerate that during a deploy window? Probably yes for a community product; absolutely not if there were money involved.

Three – staleness. Already covered, but worth noting that the user perception of staleness is shaped by the UI. If we show ‘updated 2 seconds ago’ on the leaderboard and it’s actually 8 seconds, users notice. Either be honest about staleness or hide the timestamp.

Four – admin UI. If the result is entered wrong, we recompute everything from a wrong baseline. We’d want an undo / amend path with audit trail.

Five – GDPR. When a user requests deletion, we strip them from Postgres but their score contributed to past leaderboards. We have to decide whether past leaderboards are recomputed or whether the user’s row becomes a tombstone. That’s a policy decision, not a tech one – needs legal.”

Why this phase is the most senior phase

Most candidates run out of steam after the deep dive and just answer follow-up questions. **You volunteer the risks.** That signals: I think about my own designs critically.

The five-risk list also subtly demonstrates all six categories of architecture decision (Lesson 5.1). *Stream backlog* is failure-mode design. *Cold cache* is migration thinking. *Staleness vs UI* is trade-off ownership. *Admin UI audit trail* is domain modeling. *GDPR* is a policy decision that crosses technical and legal layers.

9. Phase 7 (50–60 min) – Wrap and Q&A

What you say

“OK, summary in 30 seconds: the architecture is event-driven scoring, Redis-cached read paths, two boundaries – match service and scoring worker. The known weak points are stream backlog under burst, cold cache on Redis restart, and GDPR deletion of historic-leaderboard data. Things I haven’t covered that I’d want to: notifications fan-out, social features, and the admin tooling itself. Want to dig into any of those, or any part of what I sketched?”

Done. That summary:

- Recaps the design in three nouns.
- Names the weak points you already volunteered.
- Names what you didn’t cover (so the interviewer doesn’t think you forgot).
- Hands the floor back.

Memorize that close. The same structure works for any system-design problem.

10. The AI integration recap

Through 60 minutes of design, you reached for the AI:

- **Once or twice** for a sanity check on a number you didn't have.
- **Possibly once** to surface a feature you might have missed.
- **Zero times** to make an architectural decision.

That's the right ratio. **AI in system-design interviews is a sounding board, not a co-architect.** Anyone who lets the AI drive will get an "average AI-generated design," which interviewers see all day and can spot in 5 seconds.

11. The "may I use the AI?" opening

If the interviewer hasn't already said you can, **ask**:

"Before I start — am I allowed to use an LLM as a sounding board during this hour? I'd use it sparingly — for a sanity-check on a number, or to surface trade-offs — but not for the architectural decisions themselves."

Two outcomes:

1. They say yes — and your follow-through (asking sparingly, naming each use) impresses.
2. They say no — and you've shown you respect their format. Not a loss either way.

The phrasing matters. *"As a sounding board"*. *"Sparingly"*. *"Not for the architectural decisions"*. Each phrase signals the right mindset.

12. Common pitfalls in this format

12.1. Over-reliance on the AI

A candidate I watched once asked the AI eleven times in a 50-minute interview. Each ask was a small question. Cumulatively, the panel concluded the candidate couldn't make decisions without the AI. **Limit yourself to two or three deliberate uses.**

12.2. Drawing without narrating

Silent drawing is the worst possible mode. Even if your design is perfect, the panel can't see your reasoning. **Talk.**

12.3. Skipping the clarification

Jumping into design before clarification produces a design for the wrong problem. **Five minutes of clarification saves twenty minutes of redesign.**

12.4. Forgetting to name the pillars

Every system-design hour is a referendum on Lesson 5.1's six categories. If you finish without having explicitly named at least three of them, the panel may not have heard you cover them. **Name them as you cover them.** *"This is a boundary-placement decision..."* makes it visible.

13. CHECK YOURSELF

- ☐ Can you list the seven phases (0-5, 5-10, etc.) and what happens in each?
 - ☐ Can you deliver Phase 1's clarifying questions for any "design X" prompt?
 - ☐ Can you deliver Phase 7's wrap-up summary for any design?
 - ☐ Can you name where the AI legitimately helps in this hour and where it doesn't?
 - ☐ Can you sketch the high-level design for the fantasy app from memory?
-

14. Where you are now

You have a worked system-design walkthrough that you can adapt to any "design X" prompt. Move on to [03-trade-off-conversations.md](#) — the language and structure of trade-off discussions, which is the texture every design hour eventually demands.